

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

Отчет

по лабораторной работе №2
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Оценка времени выполнения программ».

Выполнил:
Студент группы 25BBB3
Мартьянов А.Ю.
Дмитриев С.А.

Приняли:
Юрова О.В.
Деев М.В.

Пенза 2026

Цель работы:

Получение практических навыков использования функций для измерения и оценки времени выполнения программы и ее отдельных частей.

Лабораторные задания:

Задание 1:

1. Вычислить порядок сложности программы (O -символику).
2. Оценить время выполнения программы и кода, выполняющего перемножение матриц, используя функции библиотеки `time.h` для матриц размерами от 100, 200, 400, 1000, 2000, 4000, 10000.
3. Построить график зависимости времени выполнения программы от размера матриц и сравнить полученный результат с теоретической оценкой.

Задание 2:

1. Оценить время работы каждого из реализованных алгоритмов на случайном наборе значений массива.
2. Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой возрастающую последовательность чисел.
3. Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой убывающую последовательность чисел.
4. Оценить время работы каждого из реализованных алгоритмов на массиве, одна половина которого представляет собой возрастающую последовательность чисел, а вторая, - убывающую.
5. Оценить время работы стандартной функции `qsort`, реализующей алгоритм быстрой сортировки на выше указанных наборах данных.

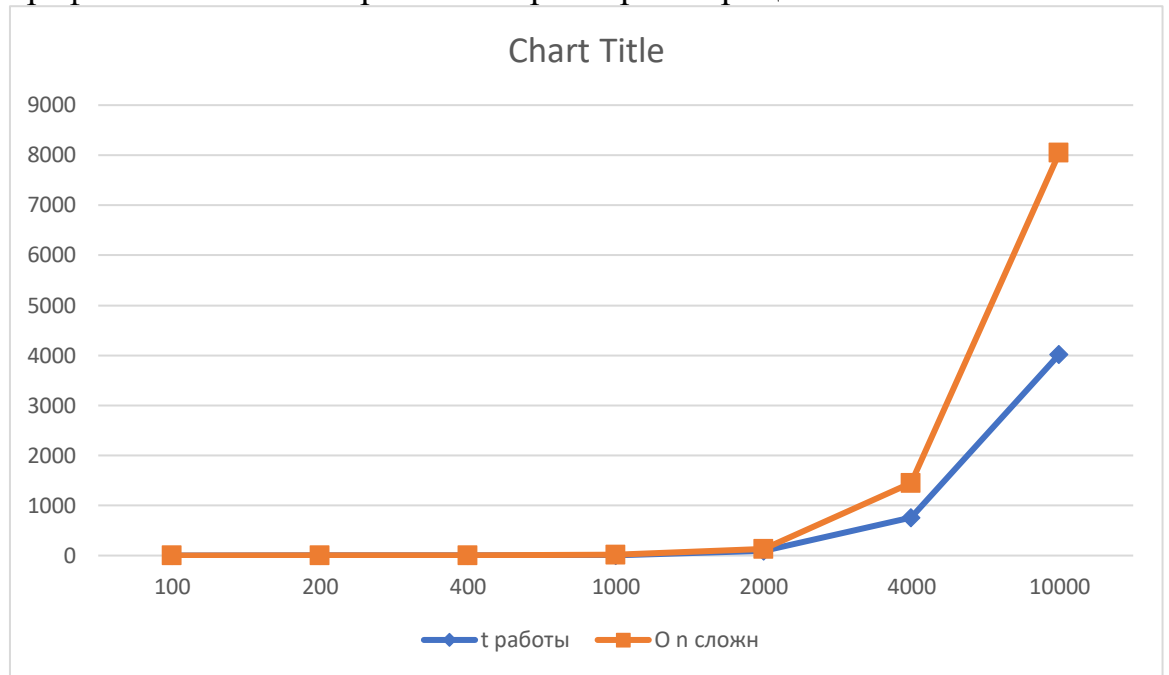
Практическая часть:

Задание 1:

1. Порядок сложности программы определили как $O(n^3)$, так как здесь реализован алгоритм перемножения 2 матриц, который реализуется 3 вложенными циклами.
- 2.

Размер матриц (эл)	t работы программы (с)	t работы умножения (с)
100	0,006	0.002
200	0,043	0.021
400	0,539	0.159
1000	9,224	2.874
2000	92,032	44.554
4000	763,708	432.111
10000	4020.043	4025.54

3. График зависимости времени от размера матриц.



Результат работы программы:

Задание 1:

```
Замер времени перемножения квадратных матриц
Размер матрицы: 100 x 100 | Время выполнения: 0,006 секунд
Размер матрицы: 200 x 200 | Время выполнения: 0,043 секунд
Размер матрицы: 400 x 400 | Время выполнения: 0,539 секунд
Размер матрицы: 1000 x 1000 | Время выполнения: 9,224 секунд
Размер матрицы: 2000 x 2000 | Время выполнения: 92,032 секунд
Размер матрицы: 4000 x 4000 | Время выполнения: 763,708 секунд
Размер матрицы: 10000 x 10000 | Время выполнения: 4020,043 секунд
Теоретическая сложность алгоритма: O(n^3)

C:\Users\silve\Desktop\лаовиз\лаба 2\ConsoleApplication1\x64\Debug\ConsoleApplication1.exe (процесс 7852) завершил работу с кодом 0 (0x0).
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" ->"Параметры" ->"Отладка" -> "Автоматически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно:
```

Рисунок 1

Задание 2:

1. Оценили время работы каждого из реализованных алгоритмов на случайном наборе значений, возрастающем, убывающем и возрастающем убывающем массива. (Массив из 5 тыс. элементов, числа от 0 до 5000).

```
=== Тестирование методов сортировки ===
```

Размер массива: 5000

N	Случайный	Возрастающий	Убывающий	Вз и уб
QuickSort	0,001000	0,058000	0,035000	0,033000
ShellSort	0,000000	0,000000	0,000000	0,001000
QSort	0,003000	0,001000	0,002000	0,002000

Рисунок 2

2. Оценили время работы каждого из реализованных алгоритмов на случайном наборе значений, возрастающем, убывающем и возрастающем убывающем массива. (Массив из 50 тыс. элементов, числа от 0 до 50000).

Размер массива: 50000

N	Случайный	Возрастающий	Убывающий	Вз и уб
QuickSort	0,006000	4,608000	3,473000	1,878000
ShellSort	0,010000	0,003000	0,004000	0,002000
QSort	0,040000	0,018000	0,017000	0,019000

Рисунок 3

1. Встроенный QSort (Библиотечный).

- **Когда он лучший:** Практически во всех сценариях, особенно на больших массивах. На размере 50 000 элементов он показал превосходную стабильность.
- **Условия:** Идеально подходит для любых типов данных (случайных, упорядоченных или частично упорядоченных). На размере 50 000 для возрастающего массива он справился за **0,018 с**, в то время как наша собственная реализация Быстрой сортировки (QuickSort) зависла на **4,608 с**. Это связано с тем, что в библиотечном qsort оптимизирован

выбор опорного элемента (pivot), что защищает его от критического падения скорости.

2. ShellSort (Сортировка Шелла).

- **Когда он лучший:** На небольших объемах данных (до 5 000) или на уже частично отсортированных массивах.
- **Условия:** На размере 5 000 он показал абсолютные нули (меньше 0,001 с), обогнав даже библиотечный QSort. Это не самый мощный алгоритм для по-настоящему огромных массивов, хотя в данном тесте он показал себя великолепно.

3. QuickSort.

- **Когда использовать:** Только для чисто случайных данных.
- **Условия:** Этот алгоритм показал отличную скорость на случайных числах (всего **0,006 с** на 50 000 элементов — быстрее всех). Но у него есть **критический недостаток**: на упорядоченных (возрастающих/убывающих) массивах его скорость падает в сотни раз (до **4,608 с**). Это классическая проблема QuickSort, когда в качестве опорного элемента выбирается крайний элемент. Алгоритм начинает совершать квадратичное число сравнений.

Вывод: При выборе метода сортировке особое внимание необходимо уделить размеру массива. Этот параметр играет ключевую роль. Мы получили практические навыки в использовании функций для измерения и оценки времени выполнения программы и ее отдельных частей.